

BÁO CÁO KỸ THUẬT TỐI GIẢN

Xây dựng chatbot AI tư vấn chọn ngành học trong 1 tuần

1. Mục tiêu phiên bản tối giản

Mục tiêu của phiên bản này là xây dựng một chatbot tư vấn chọn ngành ở mức MVP, đủ để người dùng nhập thông tin cá nhân và nhận được gợi ý ngành phù hợp trong thời gian triển khai khoảng 1 tuần.

Phiên bản này không cố gắng làm hệ thống tuyển sinh hoàn chỉnh. Thay vào đó, hệ thống chỉ tập trung vào 4 việc quan trọng nhất:

1. Người dùng nhập điểm số, chứng chỉ, tính cách, sở thích và mục tiêu nghề nghiệp.
2. Hệ thống so sánh dữ liệu người dùng với danh sách ngành có sẵn.
3. Hệ thống tính điểm phù hợp cho từng ngành.
4. AI giải thích vì sao người dùng phù hợp với các ngành được gợi ý.

Nói ngắn gọn:

User nhập dữ liệu
→ Hệ thống chấm điểm ngành
→ AI giải thích kết quả
→ Trả về top ngành phù hợp

2. Những phần nên lược bỏ trong 1 tuần đầu

Để triển khai nhanh, không nên làm quá nhiều tính năng ngay từ đầu.

Các phần nên bỏ ở MVP 1 tuần:

Thành phần	Lý do lược bỏ
RAG phức tạp	Tốn thời gian xử lý tài liệu, embedding, chunking
Vector database	Chưa cần thiết ở giai đoạn đầu
Dashboard admin	Có thể nhập dữ liệu ngành bằng file JSON trước
Đăng nhập tài khoản	MVP có thể dùng không cần login
Lưu lịch sử tư vấn	Chưa cần trong bản demo đầu tiên
Database điểm chuẩn nhiều năm	Có thể thêm sau
Tư vấn trường chi tiết	MVP chỉ tư vấn ngành trước

Thành phần	Lý do lược bỏ
Hệ thống phân quyền	Chưa cần nếu chỉ demo
Crawler dữ liệu tuyển sinh	Làm thủ công trước để tiết kiệm thời gian

Phiên bản 1 tuần nên ưu tiên chạy được, dễ demo, dễ sửa, dễ mở rộng sau.

3. Phạm vi MVP 1 tuần

3.1. Tính năng bắt buộc

MVP cần có các tính năng sau:

1. Trang nhập thông tin người dùng.
2. Danh sách ngành học mẫu từ 20-30 ngành.
3. Thuật toán chấm điểm phù hợp ngành.
4. Hiển thị top 5 ngành phù hợp nhất.
5. AI viết phần giải thích kết quả.
6. Gợi ý điểm mạnh, điểm cần cải thiện và bước tiếp theo.

3.2. Tính năng chưa cần làm

Các tính năng sau để giai đoạn sau:

1. Tư vấn trường đại học cụ thể.
2. Tính xác suất đỗ theo điểm chuẩn thật.
3. Quản trị dữ liệu ngành bằng dashboard.
4. Lưu hồ sơ nhiều người dùng.
5. RAG với tài liệu tuyển sinh.
6. Chat nhiều lượt có memory dài.

4. Tech stack tối giản

Có hai hướng triển khai. Nếu muốn nhanh nhất, nên dùng một stack duy nhất.

Phương án khuyến nghị: Next.js fullstack

Sử dụng:

Thành phần	Công nghệ
Frontend	Next.js + React
Backend API	Next.js API Route / Server Action
Ngôn ngữ	TypeScript
Styling	Tailwind CSS

Thành phần	Công nghệ
Lưu dữ liệu ngành	File JSON
AI	OpenAI API hoặc LLM API tương đương
Deploy	Vercel

Lý do chọn phương án này:

- Một project duy nhất, dễ triển khai.
- Không cần tách frontend/backend.
- Không cần database ngay từ đầu.
- Dễ deploy lên Vercel.
- Agent có thể code nhanh hơn vì cấu trúc đơn giản.

Không nên dùng FastAPI + PostgreSQL trong tuần đầu nếu mục tiêu là làm nhanh. Các công nghệ đó tốt, nhưng sẽ tăng số lượng phần cần cấu hình.

5. Kiến trúc tối giản

```

Next.js App
  ↓
Form nhập dữ liệu
  ↓
API /api/recommend
  ↓
Load majors.json
  ↓
Scoring Engine
  ↓
OpenAI Explanation
  ↓
Result Page

```

Trong phiên bản này:

- `majors.json` là database ngành tạm thời.
- `recommendation.ts` là file xử lý thuật toán chấm điểm.
- `/api/recommend` nhận dữ liệu người dùng và trả kết quả.
- AI chỉ dùng để giải thích, không dùng để tự chọn ngành.

6. Cấu trúc thư mục đề xuất

```

major-advisor-ai/
├─ app/
│  └─ page.tsx

```

```

├── result/
│   ├── page.tsx
│   └── api/
│       └── recommend/
│           └── route.ts
├── components/
│   ├── ProfileForm.tsx
│   ├── ResultCard.tsx
│   └── ScoreBreakdown.tsx
├── data/
│   └── majors.json
├── lib/
│   ├── scoring.ts
│   ├── prompt.ts
│   └── types.ts
├── .env.local
├── package.json
└── README.md

```

Đây là cấu trúc đủ đơn giản để agent có thể triển khai nhanh.

7. Dữ liệu đầu vào tối giản

Không nên hỏi quá nhiều. Chỉ cần hỏi các trường quan trọng nhất.

7.1. Form nhập liệu đề xuất

Người dùng nhập:

Trường	Kiểu dữ liệu	Bắt buộc
Điểm tổng gần nhất	number	Có
Điểm Toán	number	Có
Điểm Văn	number	Có
Điểm Anh	number	Có
IELTS	number	Không
SAT	number	Không
Tính cách MBTI/SBTI	text/select	Không
Sở thích	multi-select	Có
Mục tiêu nghề nghiệp	multi-select	Có

Trường	Kiểu dữ liệu	Bắt buộc
Không thích điều gì	multi-select	Không

7.2. Sở thích nên cho chọn sẵn

Công nghệ
 Kinh doanh
 Giao tiếp
 Sáng tạo
 Ngôn ngữ
 Du lịch
 Y tế
 Giáo dục
 Phân tích dữ liệu
 Thiết kế
 Luật
 Tài chính

7.3. Mục tiêu nghề nghiệp nên cho chọn sẵn

Thu nhập cao
 Công việc ổn định
 Môi trường quốc tế
 Công việc sáng tạo
 Làm việc với con người
 Làm việc với dữ liệu
 Có cơ hội thăng tiến
 Có thể làm freelancer/remote

8. Dữ liệu ngành tối giản

Không cần database phức tạp. Chỉ cần file `data/majors.json`.

Ví dụ:

```
[
  {
    "id": "marketing",
    "name": "Marketing",
    "category": "Kinh tế - Kinh doanh",
    "description":
      "Ngành học về nghiên cứu thị trường, xây dựng thương hiệu, truyền thông và
      phát triển khách hàng.",
    "recommendedSubjects": {
      "math": 0.5,
```

```

    "literature": 0.5,
    "english": 0.8
  },
  "interestTags": ["kinh doanh", "giao tiếp", "sáng tạo", "phân tích dữ liệu"],
  "careerGoalTags": ["thu nhập cao", "môi trường quốc tế", "công việc sáng tạo", "cơ hội thăng tiến"],
  "personalityTags": ["ENFP", "ENTJ", "ESFJ", "ESTP"],
  "skillsToImprove": ["Excel", "Content Marketing", "Phân tích dữ liệu", "Thuyết trình"],
  "careerPaths": ["Marketing Executive", "Brand Executive", "Digital Marketer", "Market Researcher"]
},
{
  "id": "computer_science",
  "name": "Công nghệ thông tin",
  "category": "Công nghệ",
  "description": "Ngành học về lập trình, hệ thống phần mềm, dữ liệu, mạng máy tính và giải quyết vấn đề bằng công nghệ.",
  "recommendedSubjects": {
    "math": 0.9,
    "literature": 0.1,
    "english": 0.5
  },
  "interestTags": ["công nghệ", "phân tích dữ liệu"],
  "careerGoalTags": ["thu nhập cao", "làm việc với dữ liệu", "có thể làm freelancer/remote"],
  "personalityTags": ["INTJ", "INTP", "ENTJ", "ISTJ"],
  "skillsToImprove": ["Lập trình", "Toán tư duy", "Tiếng Anh chuyên ngành", "Git"],
  "careerPaths": ["Software Engineer", "Data Analyst", "AI Engineer", "System Administrator"]
}
]

```

MVP nên có khoảng 20-30 ngành. Không cần quá nhiều.

Danh sách ngành nên có trước:

Công nghệ thông tin
 Khoa học dữ liệu
 Trí tuệ nhân tạo
 An toàn thông tin
 Marketing
 Quản trị kinh doanh
 Kinh doanh quốc tế
 Tài chính - Ngân hàng
 Kế toán
 Logistics
 Ngôn ngữ Anh

Ngôn ngữ Trung
Truyền thông đa phương tiện
Quan hệ công chúng
Luật
Tâm lý học
Du lịch và lữ hành
Quản trị khách sạn
Y khoa
Dược học
Điều dưỡng
Công nghệ sinh học
Kỹ thuật điện
Tự động hóa
Sư phạm Anh
Sư phạm Toán
Thiết kế đồ họa
Thương mại điện tử

9. Thuật toán chấm điểm tối giản

MVP chỉ cần 4 nhóm điểm:

Final Score =
Academic Fit x 40%
+ Interest Fit x 30%
+ Career Goal Fit x 20%
+ Personality Fit x 10%

Không cần tính Admission Chance trong tuần đầu vì muốn tính chính xác cần dữ liệu điểm chuẩn từng trường/ngành.

9.1. Academic Fit

So sánh điểm Toán, Văn, Anh của người dùng với mức yêu cầu của ngành.

Ví dụ:

Academic Fit =
Math Score x Math Requirement
+ Literature Score x Literature Requirement
+ English Score x English Requirement

Điểm môn học được chuyển từ thang 10 sang thang 100.

9.2. Interest Fit

So sánh sở thích người dùng chọn với `interestTags` của ngành.

Ví dụ:

```
Người dùng chọn: kinh doanh, giao tiếp, sáng tạo
Ngành Marketing có: kinh doanh, giao tiếp, sáng tạo, phân tích dữ liệu
→ Trùng 3/4 tag
→ Interest Fit cao
```

9.3. Career Goal Fit

So sánh mục tiêu nghề nghiệp của người dùng với `careerGoalTags` của ngành.

9.4. Personality Fit

Nếu người dùng nhập MBTI/SBTI và trùng với ngành thì cộng điểm. Nếu không nhập thì cho điểm trung lập 60/100.

Không nên để tính cách ảnh hưởng quá nhiều.

10. Code scoring tối giản

File `lib/types.ts`:

```
export type UserProfile = {
  totalScore: number;
  math: number;
  literature: number;
  english: number;
  ielts?: number | null;
  sat?: number | null;
  personality?: string | null;
  interests: string[];
  careerGoals: string[];
};

export type Major = {
  id: string;
  name: string;
  category: string;
  description: string;
  recommendedSubjects: {
    math: number;
    literature: number;
    english: number;
  };
};
```



```

};
interestTags: string[];
careerGoalTags: string[];
personalityTags: string[];
skillsToImprove: string[];
careerPaths: string[];
};

export type RecommendationResult = {
  major: Major;
  finalScore: number;
  academicFit: number;
  interestFit: number;
  careerGoalFit: number;
  personalityFit: number;
};

```

File `lib/scoring.ts`:

```

import { Major, UserProfile, RecommendationResult } from "../types";

function normalizeSubjectScore(score: number): number {
  return Math.max(0, Math.min(score * 10, 100));
}

function calculateAcademicFit(user: UserProfile, major: Major): number {
  const math = normalizeSubjectScore(user.math);
  const literature = normalizeSubjectScore(user.literature);
  const english = normalizeSubjectScore(user.english);

  const weights = major.recommendedSubjects;
  const totalWeight = weights.math + weights.literature + weights.english;

  if (totalWeight === 0) return 60;

  const score =
    math * weights.math +
    literature * weights.literature +
    english * weights.english;

  return Math.round(score / totalWeight);
}

function calculateTagFit(userTags: string[], majorTags: string[]): number {
  if (!majorTags.length) return 60;
  if (!userTags.length) return 50;

  const normalizedUserTags = userTags.map((tag) => tag.toLowerCase());
  const matched = majorTags.filter((tag) =>
    normalizedUserTags.includes(tag.toLowerCase())

```

```

    );

    return Math.round((matched.length / majorTags.length) * 100);
}

function calculatePersonalityFit(user: UserProfile, major: Major): number {
    if (!user.personality) return 60;

    const personality = user.personality.toUpperCase();
    const matched = major.personalityTags
        .map((tag) => tag.toUpperCase())
        .includes(personality);

    return matched ? 85 : 55;
}

export function recommendMajors(
    user: UserProfile,
    majors: Major[],
    limit = 5
): RecommendationResult[] {
    const results = majors.map((major) => {
        const academicFit = calculateAcademicFit(user, major);
        const interestFit = calculateTagFit(user.interests, major.interestTags);
        const careerGoalFit = calculateTagFit(user.careerGoals,
major.careerGoalTags);
        const personalityFit = calculatePersonalityFit(user, major);

        const finalScore = Math.round(
            academicFit * 0.4 +
            interestFit * 0.3 +
            careerGoalFit * 0.2 +
            personalityFit * 0.1
        );

        return {
            major,
            finalScore,
            academicFit,
            interestFit,
            careerGoalFit,
            personalityFit,
        };
    });

    return results
        .sort((a, b) => b.finalScore - a.finalScore)
        .slice(0, limit);
}

```

11. API tối giản

File `app/api/recommend/route.ts`:

```
import { NextRequest, NextResponse } from "next/server";
import majors from "@data/majors.json";
import { recommendMajors } from "@lib/scoring";
import { UserProfile, Major } from "@lib/types";

export async function POST(req: NextRequest) {
  try {
    const userProfile = (await req.json()) as UserProfile;

    const recommendations = recommendMajors(
      userProfile,
      majors as Major[],
      5
    );

    return NextResponse.json({
      recommendations,
    });
  } catch (error) {
    return NextResponse.json(
      { error: "Cannot generate recommendations" },
      { status: 500 }
    );
  }
}
```

Đây là bản chưa có AI. Sau khi scoring chạy ổn, mới thêm AI giải thích.

12. Thêm AI giải thích kết quả

AI không chọn ngành. AI chỉ giải thích kết quả đã được thuật toán tính.

12.1. Prompt tối giản

File `lib/prompt.ts`:

```
import { RecommendationResult, UserProfile } from "../types";

export function buildExplanationPrompt(
  user: UserProfile,
  results: RecommendationResult[]
) {
  return `
```

Bạn là chatbot tư vấn chọn ngành cho học sinh.
Hãy giải thích kết quả dựa trên dữ liệu đã được hệ thống chấm điểm.
Không được khẳng định tuyệt đối rằng học sinh phải chọn ngành nào.
Không được bịa điểm chuẩn hoặc tên trường nếu dữ liệu không được cung cấp.
Tính cách chỉ là yếu tố tham khảo.

Hồ sơ người dùng:

```
${JSON.stringify(user, null, 2)}
```

Kết quả chấm điểm:

```
${JSON.stringify(results, null, 2)}
```

Hãy trả lời bằng tiếng Việt theo cấu trúc:

1. Tóm tắt điểm mạnh của người dùng
 2. Top ngành phù hợp nhất
 3. Giải thích ngắn gọn từng ngành
 4. Rủi ro hoặc điểm cần cân nhắc
 5. Gợi ý 3 việc nên làm tiếp theo
- ```
`;
}
```

## 12.2. API có AI

Có thể sửa `/api/recommend/route.ts` thành:

```
import { NextRequest, NextResponse } from "next/server";
import OpenAI from "openai";
import majors from "@/data/majors.json";
import { recommendMajors } from "@/lib/scoring";
import { buildExplanationPrompt } from "@/lib/prompt";
import { UserProfile, Major } from "@/lib/types";

const openai = new OpenAI({
 apiKey: process.env.OPENAI_API_KEY,
});

export async function POST(req: NextRequest) {
 try {
 const userProfile = (await req.json()) as UserProfile;

 const recommendations = recommendMajors(
 userProfile,
 majors as Major[],
 5
);

 const prompt = buildExplanationPrompt(userProfile, recommendations);

 const completion = await openai.chat.completions.create({
 model: "gpt-4o-mini",

```

```

 messages: [
 {
 role: "system",
 content: "Bạn là cố vấn hướng nghiệp cẩn trọng, dễ hiểu và không đưa ra kết luận tuyệt đối.",
 },
 {
 role: "user",
 content: prompt,
 },
],
 });

 return NextResponse.json({
 recommendations,
 explanation: completion.choices[0]?.message?.content ?? "",
 });
} catch (error) {
 return NextResponse.json(
 { error: "Cannot generate recommendations" },
 { status: 500 }
);
}
}

```

Lưu ý: model trong code có thể thay đổi theo API bạn dùng. MVP chỉ cần một model rẻ và nhanh.

## 13. Giao diện tối giản

### 13.1. Trang chính

Trang chính chỉ cần một form.

Các input cần có:

Điểm tổng  
 Điểm Toán  
 Điểm Văn  
 Điểm Anh  
 IELTS  
 SAT  
 MBTI/SBTI  
 Sở thích  
 Mục tiêu nghề nghiệp

Sau khi submit:

• Gọi `/api/recommend`.

- Lưu kết quả vào state.
- Hiển thị kết quả ngay trên cùng trang hoặc chuyển sang `/result`.

### 13.2. Card kết quả

Mỗi ngành hiển thị:

Tên ngành  
Nhóm ngành  
Điểm phù hợp tổng  
Điểm học lực  
Điểm sở thích  
Điểm mục tiêu nghề nghiệp  
Điểm tính cách  
Mô tả ngành  
Nghề nghiệp đầu ra  
Kỹ năng nên cải thiện

### 13.3. AI explanation

Hiển thị phần giải thích AI bên dưới top ngành.

Ví dụ:

Dựa trên dữ liệu bạn cung cấp, bạn có lợi thế ở tiếng Anh, giao tiếp và tư duy kinh doanh. Các ngành phù hợp nhất là Marketing, Kinh doanh quốc tế và Truyền thông đa phương tiện...

## 14. Kế hoạch triển khai 1 tuần

### Ngày 1: Chuẩn bị project và dữ liệu ngành

Việc cần làm:

- Tạo project Next.js.
- Cài Tailwind CSS.
- Tạo cấu trúc thư mục.
- Tạo file `majors.json` với 20-30 ngành.
- Tạo type dữ liệu trong `types.ts`.

Kết quả cuối ngày:

- Project chạy được local.
- Có dữ liệu ngành mẫu.

## Ngày 2: Làm form nhập hồ sơ

Việc cần làm:

- Tạo component `ProfileForm`.
- Tạo input điểm số.
- Tạo multi-select sở thích.
- Tạo multi-select mục tiêu nghề nghiệp.
- Validate dữ liệu cơ bản.

Kết quả cuối ngày:

- Người dùng nhập được hồ sơ.
- Submit được dữ liệu ở frontend.

## Ngày 3: Làm scoring engine

Việc cần làm:

- Viết `scoring.ts`.
- Tính Academic Fit.
- Tính Interest Fit.
- Tính Career Goal Fit.
- Tính Personality Fit.
- Tính Final Score.

Kết quả cuối ngày:

- Truyền một user profile vào và nhận được top 5 ngành.

## Ngày 4: Làm API recommendation

Việc cần làm:

- Tạo `/api/recommend`.
- Load `majors.json`.
- Gọi scoring engine.
- Trả kết quả về frontend.
- Hiển thị kết quả bằng card.

Kết quả cuối ngày:

- App đã có thể tư vấn ngành không cần AI.

## Ngày 5: Tích hợp AI explanation

Việc cần làm:

- Tạo prompt giải thích.
- Tích hợp OpenAI API hoặc LLM API tương đương.

- Gửi user profile + top ngành cho AI.
- Nhận phần giải thích và hiển thị.

Kết quả cuối ngày:

- App có kết quả ngành + phần tư vấn bằng ngôn ngữ tự nhiên.

## Ngày 6: Hoàn thiện UI và kiểm thử

Việc cần làm:

- Làm UI đẹp hơn.
- Thêm loading state.
- Thêm error state.
- Test nhiều hồ sơ mẫu.
- Điều chỉnh trọng số nếu kết quả chưa hợp lý.

Kết quả cuối ngày:

- App đủ đẹp để demo.

## Ngày 7: Deploy và viết README

Việc cần làm:

- Deploy lên Vercel.
- Cấu hình biến môi trường.
- Viết README hướng dẫn chạy.
- Ghi chú giới hạn của MVP.
- Chuẩn bị demo script.

Kết quả cuối ngày:

- Có link demo online.
- Có tài liệu hướng dẫn.

---

## 15. Prompt giao việc cho coding agent

Có thể dùng prompt sau để yêu cầu agent code:

Bạn hãy xây dựng một MVP chatbot tư vấn chọn ngành học bằng Next.js, TypeScript và Tailwind CSS.

Yêu cầu chính:

1. Tạo form để người dùng nhập:
  - điểm tổng
  - điểm Toán
  - điểm Văn
  - điểm Anh



- IELTS optional
- SAT optional
- MBTI/SBTI optional
- sở thích dạng multi-select
- mục tiêu nghề nghiệp dạng multi-select

2. Tạo file data/majors.json chứa khoảng 25 ngành học.

Mỗi ngành có:

- id
- name
- category
- description
- recommendedSubjects: math, literature, english
- interestTags
- careerGoalTags
- personalityTags
- skillsToImprove
- careerPaths

3. Viết lib/scoring.ts để tính:

- academicFit
- interestFit
- careerGoalFit
- personalityFit
- finalScore

Công thức:

$$\text{finalScore} = \text{academicFit} * 0.4 + \text{interestFit} * 0.3 + \text{careerGoalFit} * 0.2 + \text{personalityFit} * 0.1$$

4. Tạo API route /api/recommend:

- nhận user profile
- load majors.json
- tính top 5 ngành phù hợp
- gọi OpenAI API để giải thích kết quả nếu có OPENAI\_API\_KEY
- nếu không có API key thì vẫn trả kết quả scoring bình thường

5. Tạo giao diện hiển thị:

- top 5 ngành
- điểm phù hợp
- breakdown điểm thành phần
- mô tả ngành
- nghề nghiệp đầu ra
- kỹ năng nên cải thiện
- phần AI explanation

6. Không dùng database.

7. Không dùng RAG.

8. Không cần login.

9. Code phải đơn giản, dễ đọc, dễ mở rộng.

10. Viết README hướng dẫn chạy local và deploy Vercel.

---

## 16. Biến môi trường

File `.env.local`:

```
OPENAI_API_KEY=your_api_key_here
```

Nếu chưa muốn dùng AI, có thể để trống và chỉ chạy scoring engine.

---

## 17. README tối giản

Nội dung README nên có:

```
Major Advisor AI MVP

Cài đặt
npm install

Chạy local
npm run dev

Cấu hình AI
Tạo file .env.local và thêm:
OPENAI_API_KEY=your_api_key_here

Tính năng
- Nhập hồ sơ học sinh
- Chấm điểm phù hợp ngành
- Gợi ý top 5 ngành
- AI giải thích kết quả

Giới hạn MVP
- Chưa có dữ liệu điểm chuẩn thật
- Chưa tư vấn trường cụ thể
- Chưa có RAG
- Chưa có đăng nhập/lưu lịch sử
```

---

## 18. Giới hạn của phiên bản 1 tuần

Cần ghi rõ với người dùng:

Kết quả tư vấn chỉ mang tính tham khảo. Hệ thống hiện chưa sử dụng dữ liệu điểm chuẩn chính thức theo từng trường và từng năm. Người dùng nên kiểm tra

lại thông tin tuyển sinh từ website chính thức của các trường trước khi ra quyết định.

Các giới hạn chính:

- Chưa tính xác suất đỗ chính xác.
- Chưa có dữ liệu tuyển sinh chính thức.
- Chưa cá nhân hóa sâu theo khu vực, học phí, trường mong muốn.
- Tính cách chỉ là yếu tố tham khảo.
- Dữ liệu ngành đang được nhập thủ công.

## 19. Sau 1 tuần nên nâng cấp gì tiếp?

Sau khi MVP chạy được, thứ tự nâng cấp nên là:

1. Thêm database Supabase để lưu ngành và hồ sơ người dùng.
2. Thêm điểm chuẩn theo trường/ngành/năm.
3. Thêm gợi ý trường an toàn, vừa sức, thử thách.
4. Thêm admin page để sửa ngành.
5. Thêm RAG từ tài liệu tuyển sinh.
6. Thêm lưu lịch sử tư vấn.
7. Thêm xuất báo cáo PDF cho học sinh.

Không nên nâng cấp tất cả cùng lúc. Mỗi lần chỉ nên thêm một lớp phức tạp.

## 20. Kết luận bản tối giản

Để triển khai trong 1 tuần cùng agent, cách hợp lý nhất là xây dựng hệ thống theo hướng:

```
Next.js fullstack
+ majors.json
+ scoring engine đơn giản
+ OpenAI explanation
+ deploy Vercel
```

Không dùng database, không dùng RAG, không cần login, không cần dashboard. Phần quan trọng nhất là làm cho scoring engine chạy ổn và kết quả tư vấn dễ hiểu.

Phiên bản này đủ để demo ý tưởng, kiểm tra phản ứng người dùng và làm nền tảng phát triển tiếp. Sau khi MVP chứng minh được giá trị, mới nên mở rộng sang Supabase, điểm chuẩn, RAG và tư vấn trường chi tiết.